SISTEME BAZATE PE CUNOȘTINȚE

Lucrare de laborator · Anul universitar 2026–2027

Laborator 2 – Înțelegerea datelor (EDA)

Înțelegerea datelor cu Python

Analiza exploratorie a datelor: statistici, analiză vizuală, corelații și verificarea calității datelor



CURS	SET DE DATE	NOTEBOOK-URI
Sisteme bazate pe cunoștințe	Calitatea vinului roșu · 1 599 de probe	Exemple rezolvate + exerciții

Cuprins

1 Analiza exploratorie a datelor 4
1.1 Setul de date 4
2 Pasul 1 – Pregătirea și prima privire 5
2.1 Biblioteci și date 5
2.2 Dimensiuni, tipuri și statistici descriptive 5
3 Pasul 2 – Ne putem baza pe date? 7
3.1 Valori lipsă 7
3.2 Rânduri duplicate 7
4 Pasul 3 – O variabilă pe rând 9
4.1 Ținta: notele de calitate 9
4.2 Distribuții: histogramă, KDE și asimetrie 10
4.3 Boxplot-uri și regula IQR 11
5 Pasul 4 – Două variabile pe rând 13
5.1 Ce atribute se leagă de calitate? 13
5.2 Un număr față de o categorie: alcoolul în funcție de calitate 14
5.3 Două numere: diagrame de dispersie 15
6 Pasul 5 – Multe variabile deodată 16
6.1 Harta termică a corelațiilor, făcută corect 16
6.2 Pair plot – pe o selecție de variabile 17
6.3 Profilurile grupurilor 18
7 Pasul 6 – Un instrument pentru prima trecere 19
8 Pasul 7 – Raportul de concluzii 20
8.1 Fișă recapitulativă 20
9 Temă 21

Acest laborator pe o pagină

DUPĂ ACEST LABORATOR VEȚI PUTEA

- să descrieți un set de date prin **dimensiuni, tipuri, statistici descriptive, asimetrie și boltire**;
- să verificați **calitatea** lui: valori lipsă, **rânduri duplicate**, valori aberante (regula IQR);
- să citiți **histograme, boxplot-uri, grafice de tip violin și diagrame de dispersie** și să spuneți în cuvinte ce arată;
- să măsurați relațiile cu **corelația Pearson și Spearman** și să le vedeți într-o hartă termică;
- să transformați o analiză în **afirmații susținute de numere**, utile unui factor de decizie.

CUM DECURGE LABORATORUL

- **Citiți** fișa: povestea, codul și figurile pe care trebuie să le obțineți.
- **Rulați** aceiași pași în `Lab2_Exemple_Rezolvate.ipynb`.
- **Exersați** în `Lab2_Exercitii.ipynb`: pinguinii din Antarctica, apoi un fișier corupt de la un birou inteligent, cu **opt defecte ascunse**.
- Fiecare sarcină are o **autoverificare** (✅ / ❌); interpretările scrise contează și ele.

Fișierele acestui laborator

Fișier	Ce conține
`Lab2_Exemple_Rezolvate.ipynb`	Analiza completă din această fișă, pas cu pas, cu casete „Ce învățăm”
`Lab2_Exercitii.ipynb`	Partea A: pinguini · Partea B: detectivul de date · autoverificări
`winequality-red.csv`	1 599 de vinuri roșii *Vinho Verde*: 11 măsurători chimice + nota de calitate
`penguins.csv`	344 de pinguini din Arhipelagul Palmer (Partea A)
`messy_office.csv`	Două zile de citiri de la un birou inteligent, cu defecte introduse intenționat (Partea B)

POVESTEA

Sunteți analistul de date al unei crame. În laborator s-a măsurat chimia a 1 599 de vinuri roșii, iar o comisie de degustători a notat fiecare vin. Managerul întreabă: **„Ce face un vin bun – și ne putem baza pe aceste date?”** Veți răspunde cu fluxul standard al analizei exploratorii.

1 Analiza exploratorie a datelor

Analiza exploratorie a datelor (EDA) este primul lucru pe care îl faceți cu un set de date, *înainte* de orice model. Studiați datele cu ajutorul statisticilor și al graficelor pentru a le înțelege trăsăturile principale, relațiile dintre variabile și problemele. EDA vă spune ce variabile contează, ce valori par suspecte și ce întrebări merită puse. În Python munca se face cu **pandas** (tabele), **NumPy** (numere), **Matplotlib** și **Seaborn** (grafice).

Pasul	Întrebarea	Instrumente
1 · Prima privire	Ce conține tabelul?	`shape`, `info()`, `describe()`
2 · Încrederea	Ne putem baza pe date?	valori lipsă, duplicate, valori aberante
3 · O variabilă	Cum este distribuită fiecare variabilă?	histogramă, asimetrie, boxplot
4 · Două variabile	Cum se leagă două variabile?	corelație, boxplot / violin, diagramă de dispersie
5 · Multe variabile	Cum interacționează mai multe variabile?	hartă termică, pair plot, profil de grup
6 · Un instrument pentru prima trecere	Poate software-ul să facă partea de rutină?	`fg-data-profiling`, `sweetviz`

1.1 Setul de date

Wine Quality (vin roșu *Vinho Verde* din nordul Portugaliei) a fost colectat de P. Cortez et al. (*Modeling wine preferences by data mining from physicochemical properties*, 2009) și este publicat în UCI Machine Learning Repository. Fiecare rând este un vin; ținta `quality` este nota dată de degustători.

Variabila	Semnificația	Unitatea
fixed acidity	acid tartric și alți acizi nevolatili	g/dm³
volatile acidity	acid acetic; în exces dă gust de oțet	g/dm³
citric acid	adaugă prospețime	g/dm³
residual sugar	zahărul rămas după fermentație	g/dm³
chlorides	sare	g/dm³
free / total sulfur dioxide	conservant SO_2: forma liberă / liber + legat	mg/dm³
density	densitatea vinului	g/cm³
pH	scara acidității (mai mic = mai acid)	–
sulphates	aditiv al vinului (sulfat de potasiu)	g/dm³
alcohol	conținutul de alcool	% vol
quality (ținta)	nota senzorială dată de degustători (numere întregi 3 … 8 în acest fișier)	notă

> **NU SCRIEȚI CĂI ABSOLUTE ÎN COD**
> O cale precum `C:\Users\<utilizator>\Desktop\…\winequality-red.csv` funcționează doar pe calculatorul vostru. Păstrați CSV-ul **lângă notebook** și folosiți o cale *relativă*, ca mai jos. Astfel codul rulează pe orice calculator (și în Colab, după încărcarea fișierului).

2 Pasul 1 – Pregătirea și prima privire

2.1 Biblioteci și date

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")        # un stil curat pentru toate figurile
```

```
df = pd.read_csv("winequality-red.csv")      # cale relativă: fișierul este lângă notebook
df.head().T                                  # transpus: încap toate cele 12 coloane
```

REZULTAT

	0	1	2	3	4
fixed acidity	7.4000	7.8000	7.800	11.200	7.4000
volatile acidity	0.7000	0.8800	0.760	0.280	0.7000
citric acid	0.0000	0.0000	0.040	0.560	0.0000
residual sugar	1.9000	2.6000	2.300	1.900	1.9000
chlorides	0.0760	0.0980	0.092	0.075	0.0760
free sulfur dioxide	11.0000	25.0000	15.000	17.000	11.0000
total sulfur dioxide	34.0000	67.0000	54.000	60.000	34.0000
density	0.9978	0.9968	0.997	0.998	0.9978
pH	3.5100	3.2000	3.260	3.160	3.5100
sulphates	0.5600	0.6800	0.650	0.580	0.5600
alcohol	9.4000	9.8000	9.800	9.800	9.4000
quality	5.0000	5.0000	5.000	6.000	5.0000

2.2 Dimensiuni, tipuri și statistici descriptive

`shape` este un **atribut** (fără paranteze): numărul de rânduri și de coloane. `info()` afișează tipul și numărul de valori nelipsă pentru fiecare coloană.

```
print("shape:", df.shape)
df.info()
```

REZULTAT

```
shape: (1599, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         1599 non-null   float64
 1   volatile acidity      1599 non-null   float64
 2   citric acid           1599 non-null   float64
 3   residual sugar        1599 non-null   float64
 4   chlorides             1599 non-null   float64
 5   free sulfur dioxide   1599 non-null   float64
 6   total sulfur dioxide  1599 non-null   float64
 7   density               1599 non-null   float64
 8   pH                    1599 non-null   float64
 9   sulphates             1599 non-null   float64
 10  alcohol               1599 non-null   float64
 11  quality               1599 non-null   int64
dtypes: float64(11), int64(1)
memory usage: 150.0 KB
```

`describe()` oferă numărul, media, abaterea standard, minimul, cuartilele și maximul. Când sunt multe coloane, `describe().T` se citește mai ușor; adăugăm **asimetria** (skewness) și **boltirea** (kurtosis, greutatea cozilor distribuției).

```
summary = df.describe().T
summary["skew"] = df.skew()
summary["kurtosis"] = df.kurt()
summary[["mean", "std", "min", "50%", "max", "skew", "kurtosis"]].round(2)
```

REZULTAT

	mean	std	min	50%	max	skew	kurtosis
fixed acidity	8.32	1.74	4.60	7.90	15.90	0.98	1.13
volatile acidity	0.53	0.18	0.12	0.52	1.58	0.67	1.23
citric acid	0.27	0.19	0.00	0.26	1.00	0.32	-0.79
residual sugar	2.54	1.41	0.90	2.20	15.50	4.54	28.62
chlorides	0.09	0.05	0.01	0.08	0.61	5.68	41.72
free sulfur dioxide	15.87	10.46	1.00	14.00	72.00	1.25	2.02
total sulfur dioxide	46.47	32.90	6.00	38.00	289.00	1.52	3.81
density	1.00	0.00	0.99	1.00	1.00	0.07	0.93
pH	3.31	0.15	2.74	3.31	4.01	0.19	0.81
sulphates	0.66	0.17	0.33	0.62	2.00	2.43	11.72
alcohol	10.42	1.07	8.40	10.20	14.90	0.86	0.20
quality	5.64	0.81	3.00	6.00	8.00	0.22	0.30

CE ÎNVĂȚĂM

1 599 de rânduri × 12 coloane, fără coloane text: 11 atribute măsurate (`float64`) și ținta `quality` (numere întregi 3 … 8). Comparați *media* cu *mediana* (50 %): pentru `residual sugar` (2,54 față de 2,2) și `chlorides` (0,087 față de 0,079) media este trasă în sus de o coadă lungă spre dreapta; maximul lui `residual sugar` este 15,5, de șapte ori mediana. Asimetria pozitivă și boltirea mare confirmă acest lucru. Variabilele au și scări foarte diferite (`density` ≈ 1, `total sulfur dioxide` până la 289), lucru important mai târziu pentru metodele bazate pe distanțe.

3 Pasul 2 – Ne putem baza pe date?

3.1 Valori lipsă

```
print(df.isnull().sum().to_string())
print("total valori lipsă:", int(df.isnull().sum().sum()))
```

REZULTAT

```
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
total valori lipsă: 0
```

3.2 Rânduri duplicate

O GREȘEALĂ FRECVENTĂ

`df.nunique()` numără **valorile distincte din fiecare coloană**; *nu* detectează rândurile duplicate. Pentru a găsi rândurile care apar de mai multe ori folosiți `df.duplicated()`.

```
print("valori distincte pe coloană (NU este o verificare a duplicatelor):")
print(df.nunique().to_string())

n_dup = df.duplicated().sum()
print(f"rânduri duplicate: {n_dup} din {len(df)} ({n_dup/len(df):.1%})")
```

REZULTAT

```
valori distincte pe coloană (NU este o verificare a duplicatelor):
fixed acidity            96
volatile acidity        143
citric acid              80
residual sugar           91
chlorides               153
free sulfur dioxide      60
total sulfur dioxide    144
density                 436
pH                       89
sulphates                96
alcohol                  65
quality                   6
rânduri duplicate: 240 din 1599 (15.0%)
```

15 % dintre rânduri sunt copii exacte ale altui rând. Este o eroare? Aici poate fi autentic (sticle din același lot, măsurate identic la precizie de două zecimale), dar pentru modelare este periculos: rândurile identice ajung și în

setul de antrenare, și în cel de test, iar rezultatele par mai bune decât sunt (**scurgere de date**, *data leakage*). Decizia trebuie să fie *explicită și documentată*. Să vedem cât schimbă:

```
df_u = df.drop_duplicates().reset_index(drop=True)      # tabelul fără duplicate
print("rânduri:", len(df), "->", len(df_u))
cmp = pd.DataFrame({"cu duplicate":
df["quality"].value_counts(normalize=True).sort_index(),
                    "fără duplicate":
df_u["quality"].value_counts(normalize=True).sort_index()})
print((cmp * 100).round(1).to_string())
print("alcool mediu:", round(df["alcohol"].mean(), 3), "->",
round(df_u["alcohol"].mean(), 3))
```

REZULTAT

```
rânduri: 1599 -> 1359
         cu duplicate  fără duplicate
quality
3                 0.6             0.7
4                 3.3             3.9
5                42.6            42.5
6                39.9            39.4
7                12.4            12.3
8                 1.1             1.3
alcool mediu: 10.423 -> 10.432
```

Proporțiile claselor și mediile se schimbă foarte puțin, deci eliminarea duplicatelor este sigură. **De acum folosim `df_u` (1 359 de rânduri).**

4 Pasul 3 – O variabilă pe rând

4.1 Ținta: notele de calitate

```
q = df_u["quality"].value_counts().sort_index()
fig, ax = plt.subplots(figsize=(7, 3.8))
bars = ax.bar(q.index.astype(str), q.values, color="#2A7F9E")
ax.bar_label(bars, labels=[f"{v}\n({v/len(df_u):.0%})" for v in q.values])
ax.set(title="Quality scores of 1 359 wines", xlabel="quality score",
ylabel="number")
plt.show()
```

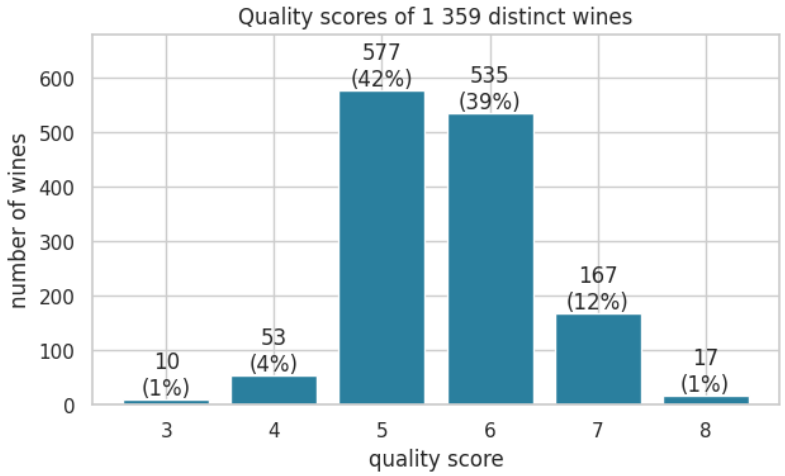


Figura 1. Distribuția notelor de calitate

CE ÎNVĂȚĂM

Notele **5 și 6 reprezintă 82 %** dintre vinuri; doar 10 vinuri au nota 3 și 17 au nota 8. Ținta este **dezechilibrată**: un model care răspunde mereu „5 sau 6” are în mare parte dreptate fără să fi învățat nimic. Când avem nevoie de o țintă mai simplă, grupăm notele în *scăzută (3–4)*, *medie (5–6)* și *ridicată (7–8)*.

```
df_u["grade"] = pd.cut(df_u["quality"], bins=[0, 4, 6, 10], labels=["low (3-4)",
"medium (5-6)", "high (7-8)"])
print(df_u["grade"].value_counts().sort_index().to_string())
```

REZULTAT

```
grade
low (3-4)         63
medium (5-6)    1112
high (7-8)       184
```

4.2 Distribuții: histogramă, KDE și asimetrie

```
features = df_u.drop(columns=["quality", "grade"]).columns.tolist()
fig, axes = plt.subplots(4, 3, figsize=(13, 11))
for ax, col in zip(axes.ravel(), features):
    sns.histplot(df_u[col], kde=True, ax=ax, color="#2A7F9E", edgecolor="white")
    ax.set_title(f"{col}\nskew = {df_u[col].skew():.2f}", fontsize=10);
ax.set_xlabel("")
axes.ravel()[-1].axis("off")
plt.tight_layout(); plt.show()
```

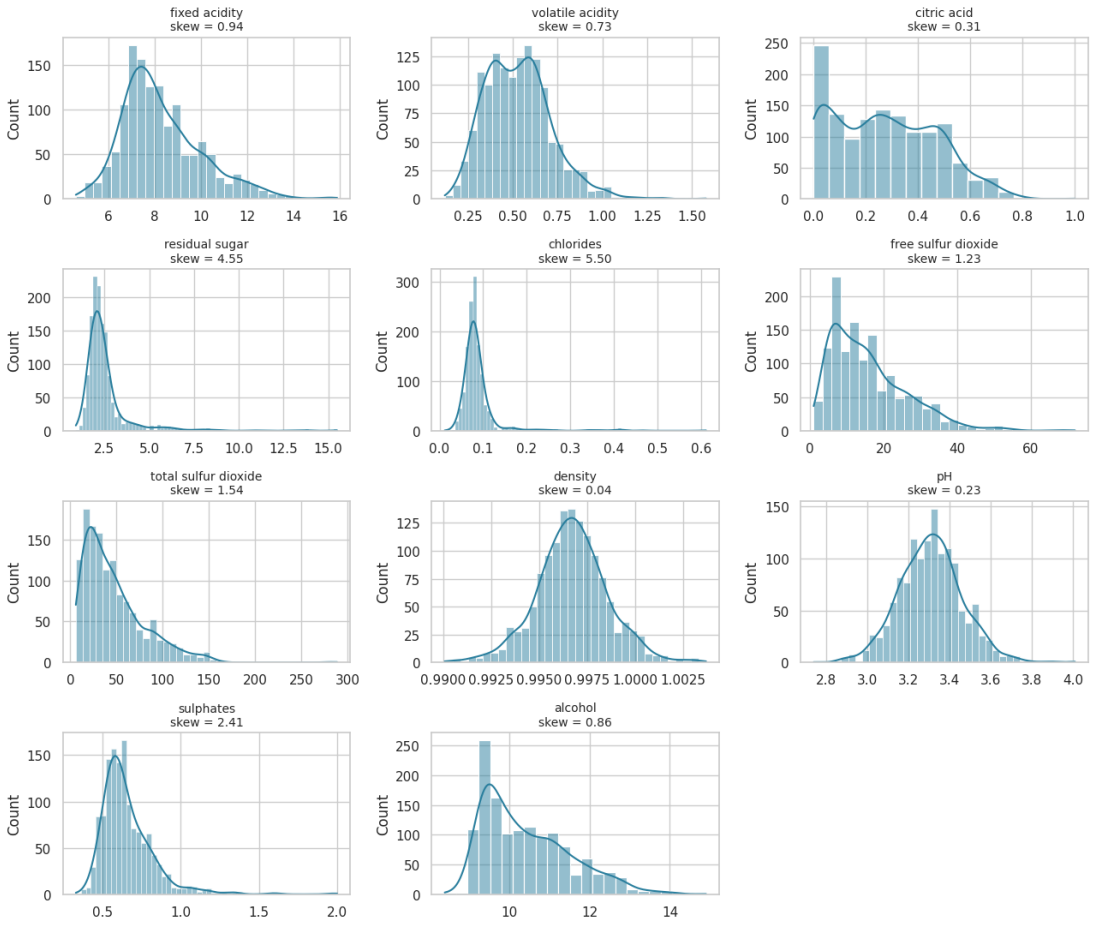


Figura 2. Histogramă cu estimarea densității nucleului (KDE) pentru fiecare dintre cele 11 atribute

Cum se citește asimetria: ≈ 0 simetrică · > 0 coadă lungă spre **dreapta** (câteva valori foarte mari) · < 0 coadă lungă spre stânga. Ca regulă practică, |skew| > 1 înseamnă asimetrie puternică.

> **CE ÎNVĂȚĂM**
> `chlorides` și `residual sugar` sunt extrem de asimetrice spre dreapta; `sulphates`, `total sulfur dioxide` și `free sulfur dioxide` sunt vizibil asimetrice; `density` și `pH` sunt aproape simetrice (în formă de clopot).

Corectarea asimetriei. O transformare logaritmică comprimă coada lungă. Ea schimbă *scara*, nu ordinea valorilor. Vedeți cât ajută în fiecare caz:

```
skewed = ["chlorides", "residual sugar", "sulphates"]
fig, axes = plt.subplots(2, 3, figsize=(12, 5.5))
for j, col in enumerate(skewed):
    sns.histplot(df_u[col], kde=True, ax=axes[0, j])
    sns.histplot(np.log1p(df_u[col]), kde=True, ax=axes[1, j])      # log(1 + x)
```

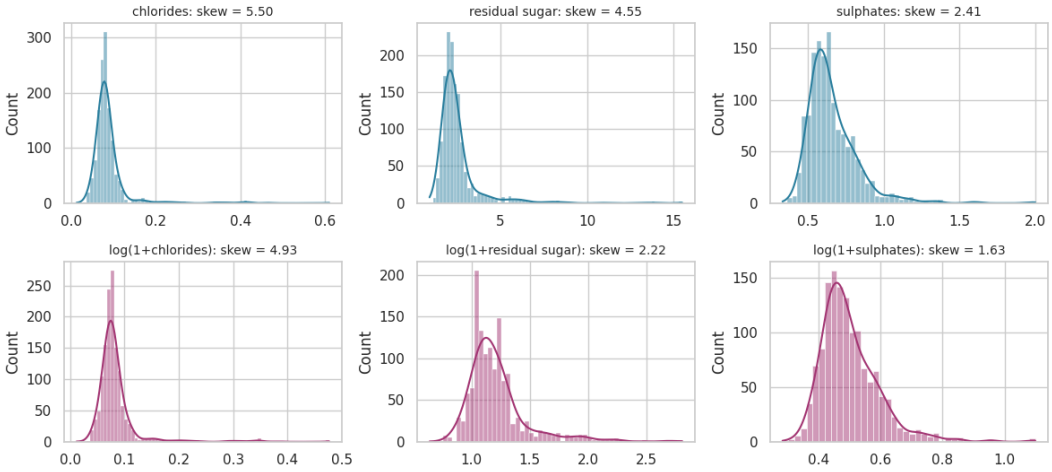


Figura 3. Sus: datele originale. Jos: după log(1 + x)

CE ÎNVĂȚĂM

Transformarea logaritmică funcționează foarte bine pentru `residual sugar` (asimetria 4,6 → 2,2) și rezonabil pentru `sulphates` (2,4 → 1,6), dar abia ajută la `chlorides` (5,5 → 4,9): câteva valori extreme (până la 0,611) rămân departe de restul. O transformare *reduce* asimetria; nu elimină punctele extreme – acelea cer o decizie separată.

4.3 Boxplot-uri și regula IQR

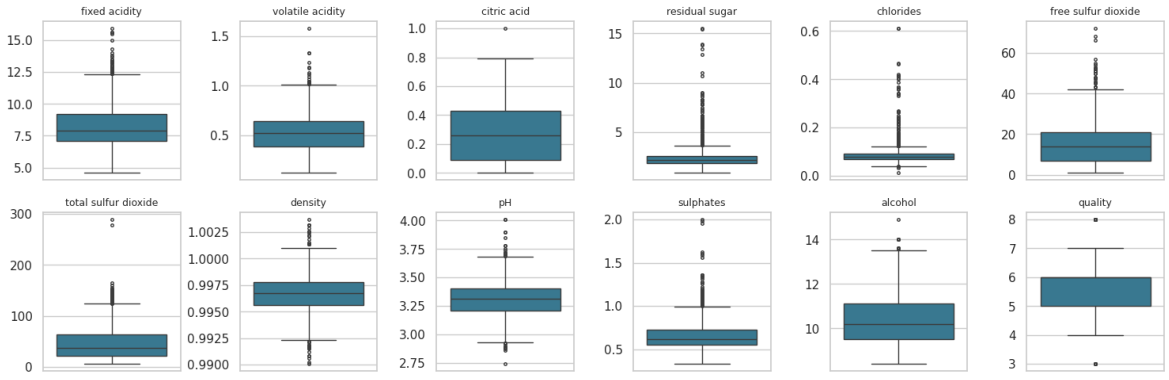


Figura 4. Boxplot pentru fiecare variabilă; punctele din afara „mustăților” sunt potențiale valori aberante

Un boxplot marchează ca valori aberante (outlieri) punctele **sub Q1 − 1,5·IQR sau peste Q3 + 1,5·IQR** (IQR = Q3 − Q1). În loc să estimați din ochi, numărați-le:

```python
def iqr_outliers(s, k=1.5):
    q1, q3 = s.quantile([.25, .75])
    lo, hi = q1 - k * (q3 - q1), q3 + k * (q3 - q1)
    n = int(((s < lo) | (s > hi)).sum())
    return pd.Series({"limita inferioară": lo, "limita superioară": hi, "nr.
aberante": n})

features = df_u.drop(columns=["quality", "grade"]).columns.tolist()
out = df_u[features].apply(iqr_outliers).T
out["% din rânduri"] = (out["nr. aberante"] / len(df_u) * 100).round(1)
out.sort_values("nr. aberante", ascending=False).round(3)
```

REZULTAT

	limita inferioară	limita superioară	nr. aberante	% din rânduri
residual sugar	0.850	3.650	126.0	9.3
chlorides	0.039	0.122	87.0	6.4
sulphates	0.280	1.000	55.0	4.0
total sulfur dioxide	-39.500	124.500	45.0	3.3
fixed acidity	3.950	12.350	41.0	3.0
density	0.992	1.001	35.0	2.6
pH	2.925	3.685	28.0	2.1
free sulfur dioxide	-14.000	42.000	26.0	1.9
volatile acidity	0.015	1.015	19.0	1.4
alcohol	7.100	13.500	12.0	0.9
citric acid	-0.420	0.940	1.0	0.1

CE ÎNVĂȚĂM

`residual sugar` (126 de puncte, 9 %) și `chlorides` (87, 6 %) au cele mai multe puncte marcate – exact variabilele puternic asimetrice: într-o distribuție asimetrică „valorile aberante” sunt coada naturală, nu erori. O valoare marcată este doar un **candidat**. Înainte de a șterge ceva, întrebați-vă: este *imposibilă*, *rară dar reală* sau o *eroare*? Aici nimic nu este fizic imposibil, așa că păstrăm toate rândurile.

5 Pasul 4 – Două variabile pe rând

5.1 Ce atribute se leagă de calitate?

```
cor = pd.DataFrame({
    "Pearson":  df_u[features].corrwith(df_u["quality"]),
    "Spearman": df_u[features].corrwith(df_u["quality"], method="spearman"),
}).sort_values("Pearson")
cor.plot.barh(figsize=(8, 5))
```

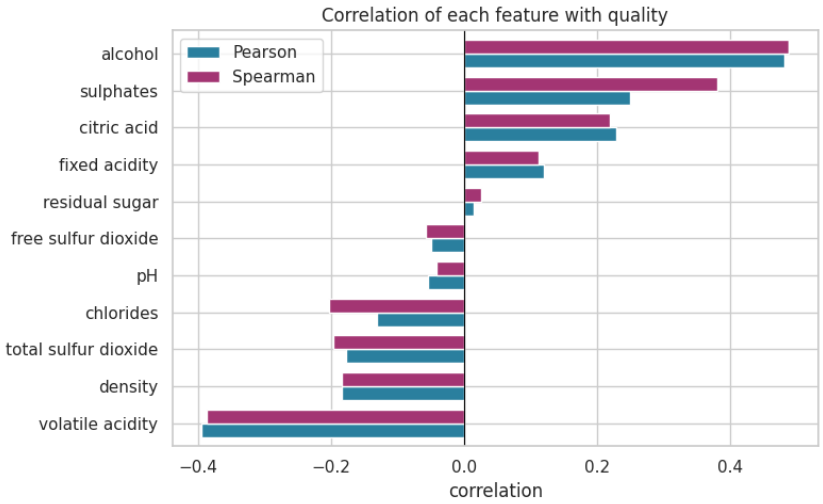


Figura 5. Corelația fiecărui atribut cu calitatea

```
features = df_u.drop(columns=["quality", "grade"]).columns.tolist()
cor = pd.DataFrame({"Pearson": df_u[features].corrwith(df_u["quality"]),
                    "Spearman": df_u[features].corrwith(df_u["quality"],
method="spearman")}).sort_values("Pearson")
cor.round(3)
```

REZULTAT

	Pearson	Spearman
volatile acidity	-0.395	-0.387
density	-0.184	-0.184
total sulfur dioxide	-0.178	-0.197
chlorides	-0.131	-0.204
pH	-0.055	-0.043
free sulfur dioxide	-0.050	-0.059
residual sugar	0.014	0.026
fixed acidity	0.119	0.112
citric acid	0.228	0.219
sulphates	0.249	0.381
alcohol	0.480	0.488

Pearson măsoară asocierea *liniară*; **Spearman** măsoară asocierea dintre *ranguri* (monotonă, robustă la valori aberante și la asimetrie). Când cei doi coeficienți sunt de acord, concluzia este solidă.

CE ÎNVĂȚĂM

Alcoolul (+0,48) este cel mai puternic corelat pozitiv cu calitatea, iar **volatile acidity (−0,40)** cel mai puternic corelat negativ (acidul acetic dă gust de oțet), urmate de sulphates (+0,25), citric acid (+0,23), total sulfur dioxide (−0,18) și density (−0,18). `residual sugar`, `free sulfur dioxide` și `pH` sunt aproape necorelate cu calitatea ($|r| < 0{,}06$). Pentru **sulphates** și **chlorides**, Spearman este clar mai mare în valoare absolută decât Pearson (0,38 față de 0,25; −0,20 față de −0,13): relația este monotonă, dar perturbată de valorile aberante, care afectează mai mult Pearson. **Corelația nu implică cauzalitate**, iar chiar și $|r| = 0{,}48$ explică doar aproximativ 23 % din varianță ($r^2 \approx 0{,}23$).

5.2 Un număr față de o categorie: alcoolul în funcție de calitate

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5), sharey=True)
sns.boxplot(data=df_u, x="quality", y="alcohol", ax=axes[0], color="#2A7F9E")
sns.violinplot(data=df_u, x="quality", y="alcohol", ax=axes[1], color="#A33573",
inner="quartile", cut=0)
```

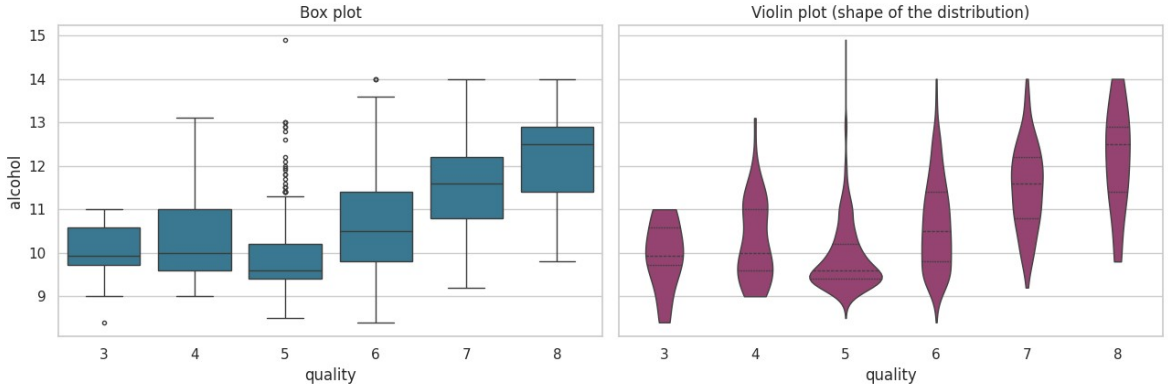


Figura 6. Conținutul de alcool pentru fiecare notă de calitate: boxplot (stânga) și grafic violin (dreapta)

```
df_u.groupby("quality")["alcohol"].agg(["count", "median", "mean"]).round(2)
```

REZULTAT

quality	count	median	mean
3	10	9.93	9.96
4	53	10.00	10.27
5	577	9.60	9.89
6	535	10.50	10.66
7	167	11.60	11.49
8	17	12.50	12.16

Cum se citește	Boxplot	Grafic violin
Centrul	linia din interiorul casetei = mediana	linia / liniile punctate = mediana și cuartilele
Împrăștierea	caseta = IQR (50 % din mijloc); o casetă lungă = variabilitate mare	lățimea formei = densitatea valorilor
Extremele	mustățile și punctele = valori aberante	cozi lungi și subțiri
Forma	ascunsă	umflăturile arată valorile cele mai frecvente; două umflături = două grupuri

CE ÎNVĂȚĂM

De la nota 5 în sus, mediana alcoolului crește constant: **9,6 % → 10,5 % → 11,6 % → 12,5 %** pentru notele 5, 6, 7, 8. Notele foarte rare 3 și 4 (10 și 53 de vinuri) nu urmează tendința (mediana ≈ 10 %), deci relația nu este perfect monotonă. Vinurile cu nota 5 sunt strâns concentrate în jurul valorii de 9,5 %, cu o coadă lungă în sus (valorile aberante din boxplot); vinurile cu nota 6 sunt larg împrăștiate între 9 și 14 %: multe vinuri de nota 6 seamănă cu cele de nota 5, iar altele cu cele de nota 7.

SFAT

Un *swarm plot* cu 1 359 de puncte arată bine pentru date puțin numeroase, dar este lent, iar Seaborn avertizează că multe puncte nu pot fi plasate. Pentru eșantioane mai mari preferați boxplot-ul sau graficul violin (sau `stripplot` cu `alpha`).

5.3 Două numere: diagrame de dispersie

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
sns.scatterplot(data=df_u, x="alcohol", y="volatile acidity", hue="grade", alpha=.65,
ax=axes[0])
sns.regplot(data=df_u, x="density", y="fixed acidity", scatter_kws={"alpha": .35},
ax=axes[1])
```

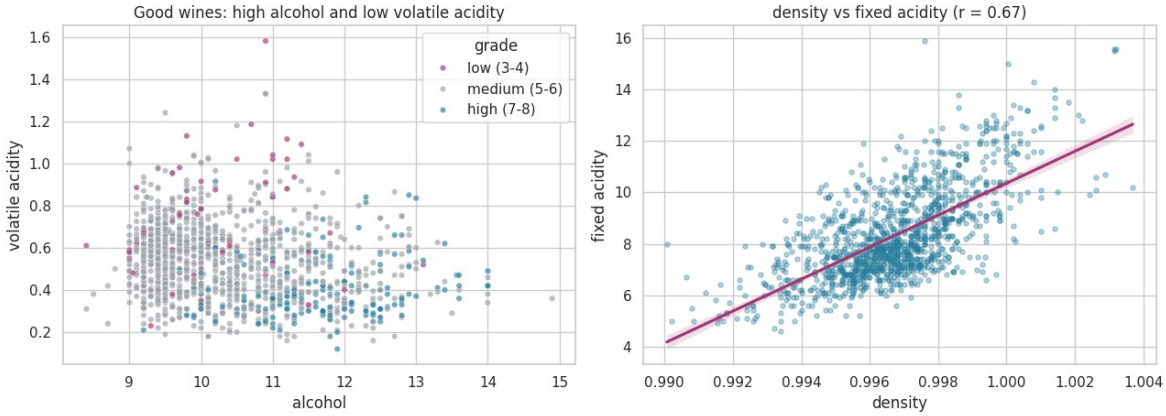


Figura 7. Stânga: două atribute separă grupurile de calitate. Dreapta: density și fixed acidity variază împreună

CE ÎNVĂȚĂM

În graficul din stânga, vinurile de calitate ridicată (albastru) sunt în **dreapta jos** (alcool peste ~11 %, volatile acidity sub ~0,6), iar cele de calitate scăzută (roz) sunt mai ales **sus** (volatile acidity peste ~0,8) la orice nivel de alcool: două atribute luate împreună spun mai mult decât fiecare separat, iar grupul mediu, gri, se suprapune peste ambele. În graficul din dreapta, `density` și `fixed acidity` sunt puternic legate (r = 0,67): conțin informații care se suprapun (**multicoliniaritate**).

6 Pasul 5 – Multe variabile deodată

6.1 Harta termică a corelațiilor, făcută corect

O matrice completă de corelații este simetrică, așa că afișați doar un triunghi și folosiți o **scară de culori divergentă, centrată în 0**, astfel încât să se vadă *semnul*.

```
corr = df_u[features + ["quality"]].corr()
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)         # ascunde triunghiul
superior
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="vlag", center=0, vmin=-1,
vmax=1,
            square=True, linewidths=.5)
```

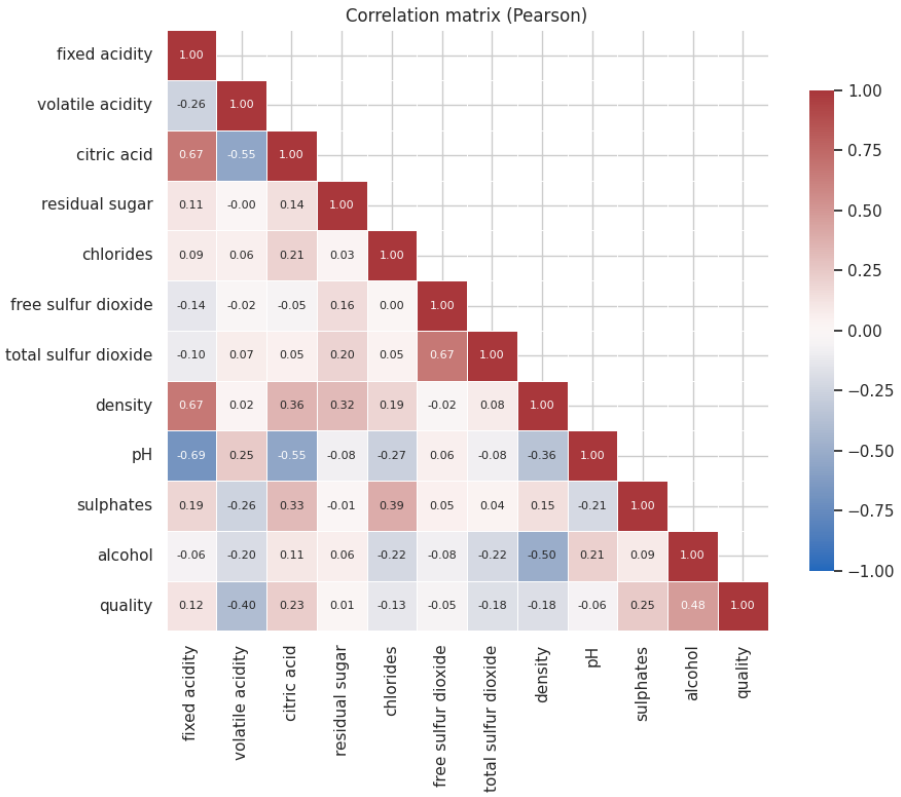


Figura 8. Matricea de corelații (Pearson), triunghiul inferior

```
corr = df_u[features + ["quality"]].corr()
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
pairs = corr.where(~mask & (corr != 1)).stack().rename("r").reset_index()
pairs.columns = ["atribut 1", "atribut 2", "r"]
pairs.loc[pairs["r"].abs() > 0.6].sort_values("r", key=abs, ascending=False).round(2)
```

REZULTAT

	atribut 1	atribut 2	r
28	pH	fixed acidity	-0.69
21	density	fixed acidity	0.67
1	citric acid	fixed acidity	0.67
20	total sulfur dioxide	free sulfur dioxide	0.67

CE ÎNVĂȚĂM

Cele mai puternice perechi sunt **fixed acidity ↔ pH (−0,69)**, **fixed acidity ↔ citric acid (+0,67)**, **fixed acidity ↔ density (+0,67)** și **free ↔ total sulfur dioxide (+0,67)**. Din punct de vedere chimic este evident (acizii scad pH-ul), dar important pentru modelare: intrările puternic corelate fac instabili coeficienții modelelor liniare.

6.2 Pair plot – pe o selecție de variabile

```
sel = ["alcohol", "volatile acidity", "sulphates", "citric acid"]
g = sns.pairplot(df_u, vars=sel, hue="grade", corner=True, diag_kind="kde",
                 diag_kws={"common_norm": False}, plot_kws={"alpha": .5, "s": 14})
```

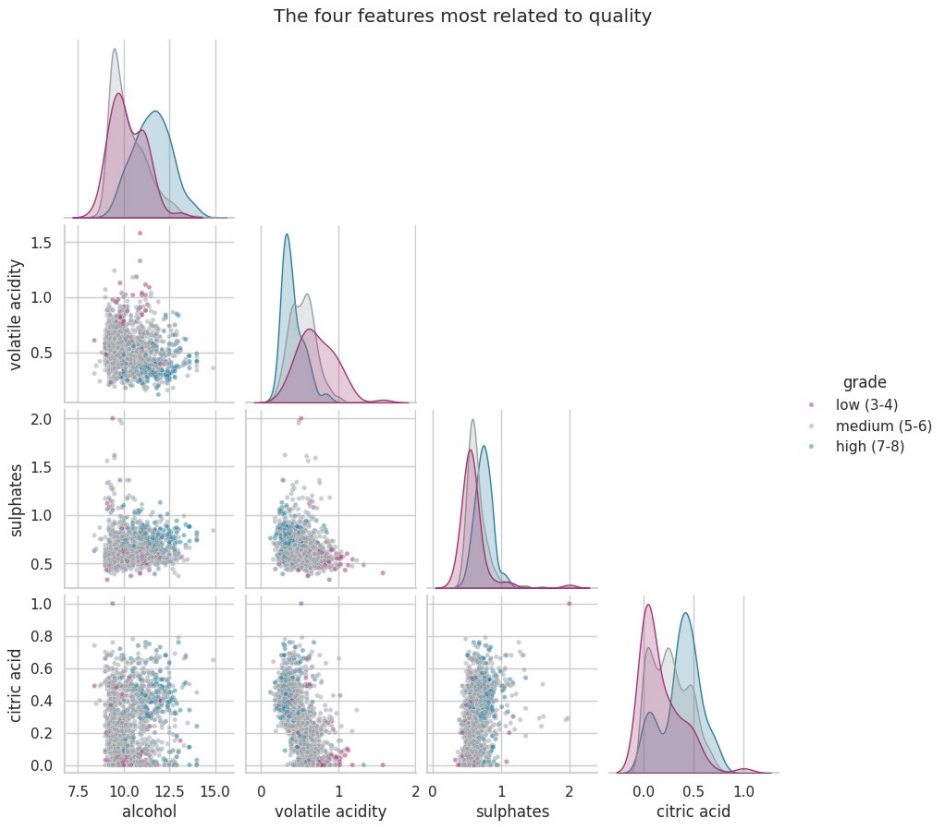


Figura 9. Pair plot al celor patru atribute cele mai legate de calitate, colorat după grupul de calitate

Cum se citește: diagonala arată distribuția fiecărei variabile (câte o curbă pentru fiecare grup); fiecare altă celulă arată o *pereche* de variabile. Cu toate cele 12 variabile, un pair plot are 144 de panouri și nu mai poate fi citit, așa că **alegeți mai întâi variabilele** (de exemplu cele mai puternic corelate, găsite în secțiunea 5.1).

CE ÎNVĂȚĂM

Pe diagonală (fiecare grup este normalizat separat, ca grupurile mici să rămână vizibile), curba alcoolului pentru grupul *high* este clar deplasată spre dreapta, iar cea a lui volatile acidity spre stânga. Niciun atribut nu separă perfect grupurile – norii de puncte se suprapun – deci predicția calității vinului va fi o sarcină imperfectă chiar și cu un model bun.

6.3 Profilurile grupurilor

Cum arată un vin tipic de calitate scăzută / medie / ridicată? Standardizați fiecare atribut (media 0, abaterea standard 1) și faceți media în interiorul fiecărui grup.

```
z = (df_u[features] - df_u[features].mean()) / df_u[features].std()
profile = z.groupby(df_u["grade"], observed=True).mean().T
sns.heatmap(profile, annot=True, fmt=".2f", cmap="vlag", center=0)
```

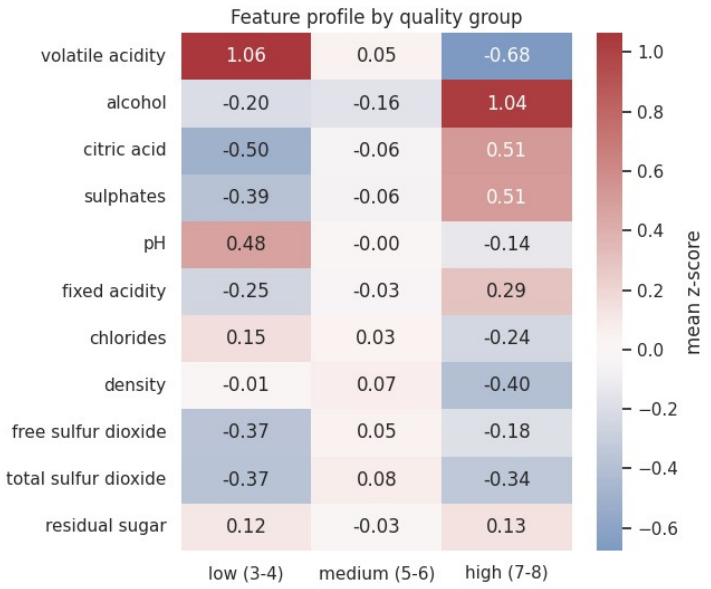


Figura 10. Valoarea standardizată medie a fiecărui atribut în fiecare grup de calitate

CE ÎNVĂȚĂM

Vinurile de calitate ridicată au alcoolul cu +1,0 abateri standard peste medie, sulphates și citric acid cu aproximativ +0,5 și volatile acidity cu −0,7. **Vinurile de calitate scăzută** se caracterizează în principal prin volatile acidity +1,1, citric acid scăzut (−0,5) și pH mai mare (+0,5) – dar *nu* prin alcool scăzut (doar −0,2). Așadar „bun” înseamnă mai ales *alcool ridicat*, iar „prost” înseamnă mai ales *oțet*. O singură imagine rezumă întreaga analiză.

7 Pasul 6 – Un instrument pentru prima trecere

Biblioteci precum **fg-data-profiling** (fostul *pandas-profiling*, apoi *ydata-profiling*) și **sweetviz** produc dintr-o singură linie un raport HTML complet – distribuții, valori lipsă, duplicate, corelații, alerte. Folosiți-le pentru a **începe** o analiză, niciodată pentru a înlocui propria voastră lectură.

NUME NOU: FG-DATA-PROFILING

Pachetul *ydata-profiling* a fost redenumit. Instalați-l cu `pip install fg-data-profiling` și importați-l ca `data_profiling`; vechiul pachet nu mai primește actualizări și afișează un avertisment de depreciere. *sweetviz* (versiunea 2.3) încă funcționează.

INSTALARE

```
%pip install fg-data-profiling sweetviz        # o singură dată, în prima celulă a notebook-ului
```

RAPORTUL DINTR-O LINIE

```
from data_profiling import ProfileReport

report = ProfileReport(df, title="Red wine quality", minimal=True,
progress_bar=False)
report.to_file("wine_profile.html")            # deschideți fișierul într-un browser
# report.to_notebook_iframe()                  # sau afișați-l în Jupyter
```

REȚINEȚI

Verificați dacă raportul **se potrivește cu propriile voastre concluzii**: el semnalează cele 240 de rânduri duplicate, variabilele puternic asimetrice și perechile puternic corelate. Dacă importul eșuează pe o versiune foarte nouă de Python, folosiți un mediu cu o versiune de Python acceptată de pachete.

8 Pasul 7 – Raportul de concluzii

O analiză bună se încheie cu **afirmații pe care un manager le poate folosi, fiecare susținută de un număr**. Le generăm din date, în loc să le scriem de mână, astfel încât rămân corecte dacă datele se schimbă.

```
top_pos = cor["Pearson"].idxmax(); top_neg = cor["Pearson"].idxmin()
share_56 = df_u["quality"].isin([5, 6]).mean()
skew_top = df_u[features].skew().abs().sort_values(ascending=False).head(2)
r_pos, r_neg = cor.loc[top_pos, "Pearson"], cor.loc[top_neg, "Pearson"]
print(f"1. {len(df)} rânduri, {n_dup} ({n_dup/len(df):.0%}) duplicate -> se folosesc
{len(df_u)}.")
print("2. Valori lipsă: niciuna.")
print(f"3. Țintă dezechilibrată: {share_56:.0%} dintre vinuri au nota 5 sau 6.")
print(f"4. Cea mai puternică corelație pozitivă: {top_pos} (r = {r_pos:+.2f}).")
print(f"5. Cea mai puternică corelație negativă: {top_neg} (r = {r_neg:+.2f}).")
print("6. Cele mai asimetrice:", ", ".join(skew_top.index))
```

REZULTAT

```
1. 1599 rânduri, 240 (15%) duplicate -> se folosesc 1359.
2. Valori lipsă: niciuna.
3. Țintă dezechilibrată: 82% dintre vinuri au nota 5 sau 6.
4. Cea mai puternică corelație pozitivă: alcohol (r = +0.48).
5. Cea mai puternică corelație negativă: volatile acidity (r = -0.40).
6. Cele mai asimetrice: chlorides, residual sugar
```

8.1 Fișă recapitulativă

Întrebarea	Instrumentul	Capcana evitată
dimensiunea tabelului, tipuri	`shape`, `info()`, `describe().T`	`shape` nu are `()`
valori lipsă	`isnull().sum()`	–
rânduri duplicate	`duplicated().sum()`, `drop_duplicates()`	`nunique()` nu verifică duplicatele
distribuția	histogramă + KDE, `skew()`, `kurt()`	cozile asimetrice nu sunt automat erori
valori aberante	boxplot, limitele IQR	decideți: eroare, rară dar reală sau coadă naturală
relația cu ținta	`corrwith`, Pearson vs Spearman, boxplot / violin	corelație ≠ cauzalitate
multe variabile	hartă termică mascată, pair plot *selectat*, profil de grup	pair plot-uri 12 × 12 ilizibile
raport rapid	`fg-data-profiling`, `sweetviz`	un început, nu răspunsul

Lecturi suplimentare

- GeeksforGeeks – EDA, Exploratory Data Analysis in Python
- Tutorialul Seaborn · Ghidul de utilizare pandas · Documentația Matplotlib
- UCI Machine Learning Repository – Wine Quality

9 Temă

Deschideți **`Lab2_Exercitii.ipynb`**. Aplicați același flux de lucru pe **două seturi de date noi**. Fiecare sarcină are un enunț, o celulă de cod pentru soluția voastră și o **autoverificare**; valorile așteptate sunt stocate sub formă de amprente, deci nu puteți citi răspunsurile – trebuie să le calculați (numerele se compară după rotunjirea la 2 zecimale).

A Partea A – Pinguinii (ghidat)

344 de pinguini din trei specii, de pe trei insule din Antarctica: lungimea și grosimea ciocului, lungimea aripioarei, masa corporală. Un set de date real, cu valori lipsă reale – și o surpriză.

Sarcina	Tema	Nivel
A1	Prima privire: dimensiuni, tipuri, celule lipsă	★☆☆
A2	Duplicate și rânduri cu valori lipsă; când este o idee proastă să le eliminăm?	★☆☆
A3	Distribuții și echilibrul speciilor	★★☆
A4	Boxplot-uri pe grupuri și regula IQR	★★☆
A5	Lungimea față de grosimea ciocului – și paradoxul lui Simpson	★★★
A6	Hartă termică, pair plot și trei concluzii	★★☆

B Partea B – Detectivul de date

`messy_office.csv` conține două zile de citiri pe minut de la un alt birou inteligent. În el sunt ascunse **opt defecte**: rânduri duplicate, valori lipsă și marcaje de timp lipsă, valori imposibile (o eroare de unitate, un cod pentru valoare lipsă, număr negativ de persoane), un senzor înghețat și vârfuri izolate. Găsiți-le și numărați-le, scrieți un **raport de defecte**, apoi faceți curățarea minimă, care nu necesită modelare. *Diagnosticați, nu reparați încă:* repararea valorilor lipsă și a zgomotului este subiectul cursurilor următoare.

> **CE PREDAȚI**
> `Lab2_Exercitii.ipynb` după *Kernel → Restart & Run All*: toate autoverificările verzi, graficele voastre, **interpretările scrise** (celule markdown) și raportul de defecte. Încărcați-l pe platforma cursului.

> **SFAT**
> Bonus opțional: rulați un raport de profilare pe `messy_office.csv`. Pe care dintre cele opt defecte le semnalează automat și pe care le ratează?